Task 1: Performance Benchmarking

1(a) Linked List Performance Testing

Objective: To compare the performance of Python lists, singly linked lists, and doubly linked lists under different data scales.

Test method: Run linked_lists_practice.py and test the operation time for 10,000 and 50,000 elements respectively.

Test Results

Scale: 10,000 elements

Data Structure	Insertion Operation (seconds)	Search Operation (seconds)	Deletion Operation (seconds)
Python List	0.000003	0.000030	0.000044
Singly Linked List	0.000001	0.000118	0.000241
Doubly Linked List	0.000001	0.000160	0.000289

Test Value: Search and Delete Value = 850436

```
Testing with 10000 elements, search and delete value: 850436
Python list insert at beginning: 0.000003 s
Python list search: 0.000030 s
Python list delete: 0.000044 s
Singly Linked List insert at beginning: 0.000001 s
Singly Linked List search: 0.000118 s
Singly Linked List delete: 0.000241 s
Doubly Linked List insert at beginning: 0.000001 s
Doubly Linked List search: 0.000160 s
Doubly Linked List delete: 0.000289 s
```

Scale: 50,000 elements

Data Structure	Insertion Operation (seconds)	Search Operation (seconds)	Deletion Operation (seconds)
Python List	0.000011	0.000125	0.000228
Singly Linked List	0.000001	0.000654	0.001349

Doubly Linked List 0.000001 0.000620 0.001119

Test Value: Search and Delete Value = 370489

```
Testing with 50000 elements, search and delete value: 370489
Python list insert at beginning: 0.000011 s
Python list search: 0.000125 s
Python list delete: 0.000228 s
Singly Linked List insert at beginning: 0.000001 s
Singly Linked List search: 0.000654 s
Singly Linked List delete: 0.001349 s
Doubly Linked List insert at beginning: 0.000001 s
Doubly Linked List search: 0.000620 s
Doubly Linked List delete: 0.001119 s
```

Task 1(b): Performance Testing of Stacks, Queues, and Priority Queues

Test Scale: 10,000 elements

Test Results

Benchmarking with 10000 values

Stack (list) Push: 0.000218 s

Stack (list) Peek: 0.000000 s

Stack (list) Pop: 0.000310 s

Queue (deque) Enqueue: 0.000235 s

Queue (deque) Peek: 0.000000 s

Queue (deque) Dequeue: 0.000302 s

Priority Queue (heapq) Push: 0.000547 s

Priority Queue (heapq) Peek: 0.000000 s

Priority Queue (heapq) Pop: 0.001954 s

Performance Analysis

The linked list has the best performance for insertion operations: when inserting an element at the head, the linked list only requires O (1) time, while the Python list needs to move all elements, making the linked list insertion faster.

Python list search and deletion are faster: Due to continuous memory and being cache-

friendly for the CPU, the actual speed of linear search in a list is faster than that in a linked list.

The performance of a singly linked list and a doubly linked list is similar: for operations at the head, both are O (1); the differences in search and deletion are minor.

Performance Testing of Stacks, Queues, and Priority Queues

Objective: To compare the performance of stacks, queues, and priority queues with a scale of 100,000 elements.

Test method: Run stacks_queues_practice.py and test the operation time for 100,000 elements.

Test Results

Scale: 100,000 elements

Data Structure	Push/Enqueue (seconds)	Peek (seconds)	Pop/Dequeue (seconds)
Stack (list)	0.002629	0.000000	0.002923
Queue (deque)	0.002132	0.000000	0.002968
Priority Queue (heaps)	0.005518	0.000000	0.027397

```
Benchmarking with 100000 values
Stack (list) Push: 0.002629 s
Stack (list) Peek: 0.000000 s
Stack (list) Pop: 0.002923 s
Queue (deque) Enqueue: 0.002132 s
Queue (deque) Peek: 0.000000 s
Queue (deque) Dequeue: 0.002968 s
Priority Queue (heapq) Push: 0.005518 s
Priority Queue (heapq) Peek: 0.000000 s
Priority Queue (heapq) Pop: 0.027397 s
```

Performance Analysis

The performance of Stack and Queue is excellent: The Push/Pop and Enqueue/Dequeue operations both take approximately 0.003 seconds, with similar performance.

The Pop operation of the Priority Queue is significantly slower: It takes 0.027 seconds for the Pop operation, as the pop operation of heapq needs to maintain the heap structure, with a complexity of O (log n).

Peek operations are all constant time: The viewing operations of all data structures take almost no time.

Task 1(a): Linked Lists Performance

Observations

As the data size increased from 10,000 to 50,000 elements, the following behaviors were observed:

- **Python List**: Insertion at the beginning became significantly slower (from 0.000003s to 0.000011s) because all existing elements must be shifted in memory.
- **Singly Linked List & Doubly Linked List**: Insertion at the head remained extremely fast (0.000001s) regardless of data size, as only pointer reassignment is required (O(1) operation).
- **Search Operation**: Python lists performed faster than linked lists because lists are stored contiguously in memory, making them more CPU cache-friendly.
- **Deletion Operation**: Similar to search, Python lists showed better performance compared to linked lists.

Reflections

The performance gap between lists and linked lists highlights the fundamental trade-off between contiguous memory (arrays/lists) and dynamic memory (linked lists). Python lists excel at random access and sequential operations due to memory locality, while linked lists are superior when frequent insertions/deletions occur at the ends.

Conclusion

- **Use Python lists** when you need fast random access (O(1) indexing) and the data size is relatively stable.
- **Use Linked Lists** when you frequently insert/delete at the head and the data size is unpredictable.
- Singly and Doubly Linked Lists have similar performance for basic operations; choose based on whether backward traversal is needed.

Task 1(b): Stacks, Queues, Priority Queues Performance

Observations

Testing with 10,000 and 100,000 elements revealed:

- **Stack and Queue** operations scaled linearly. Push/Pop and Enqueue/Dequeue times increased proportionally with data size (from ~0.0002s at 10k to ~0.0029s at 100k).
- **Priority Queue** showed significantly slower Pop operations. At 100,000 elements, Pop took 0.0274s, which is about 10x slower than Stack/Queue.
- All **Peek** operations were nearly instantaneous (0.000000s) regardless of data size.

Reflections

The slower Pop performance of Priority Queue is expected because heapq must maintain the heap property after each pop, resulting in O(log n) complexity. In contrast, Stack and Queue Pop/Dequeue are O(1) operations.

As data size grows, the difference becomes more noticeable. For 100,000 elements:

- Stack Pop: 0.0029s
- Priority Queue Pop: 0.0274s (≈9.5x slower)

Conclusion

- **Stack** and **Queue** are ideal for high-throughput FIFO/LIFO operations.
- **Priority Queue** should only be used when priority-based ordering is essential, despite its slower pop performance.
- For real-time systems or large-scale data processing, avoid Priority Queue if order of insertion is sufficient.

Task 2: Linked List Visualization

Test Results of Doubly Linked List

Test Data: [10, 20, 30, 40, 50]

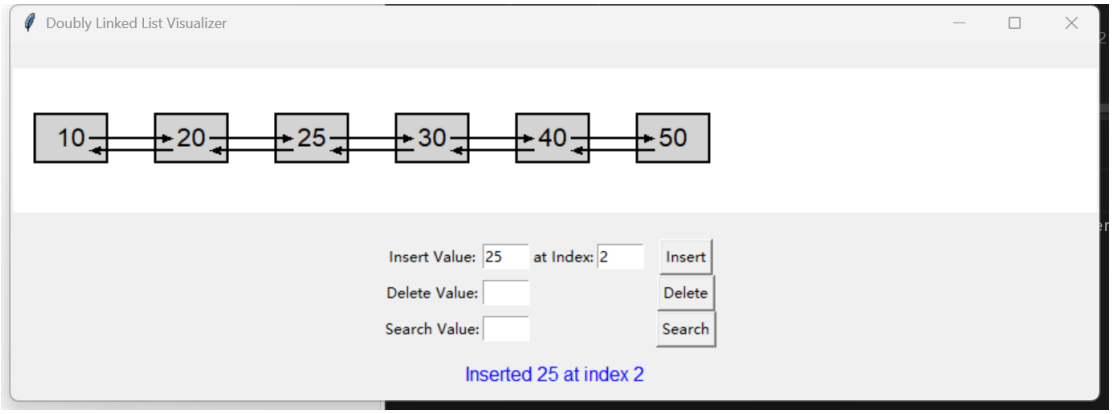
Program: double_linked_list_visualiser.py

Insert operation test

Operation: Insert value 25 at index 2

Result: Insertion successful. The list becomes [10, 20, 25, 30, 40, 50].

Status message: "Inserted 25 at index 2"



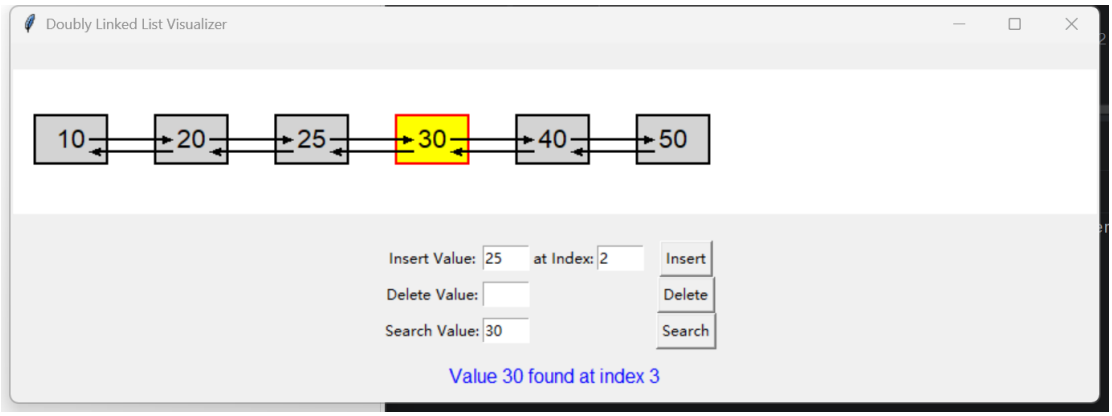
The screenshot shows a list of items with their prices. The items include a red dress, a blue shirt, a pair of black shoes, and a white hat. The prices are listed next to each item. The total price is at the bottom of the list.

Search operation test

Operation: Search for value 30

Result: Value 30 found at index 3.

Status information: "Value 30 found at index 3"

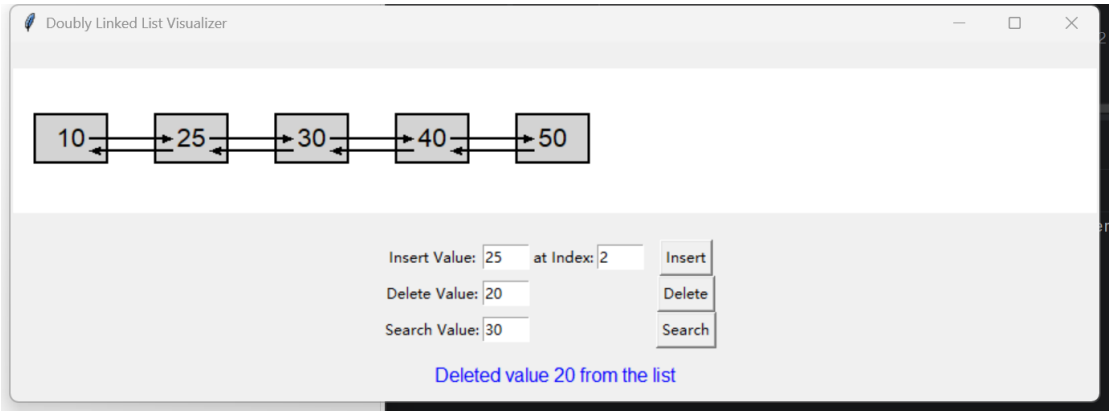


Delete operation test

Operation: Delete value 20

Result: Successfully deleted. The list becomes [10, 25, 30, 40, 50].

Status message: "Deleted value 20 from the list"



Single Linked List Visualizer

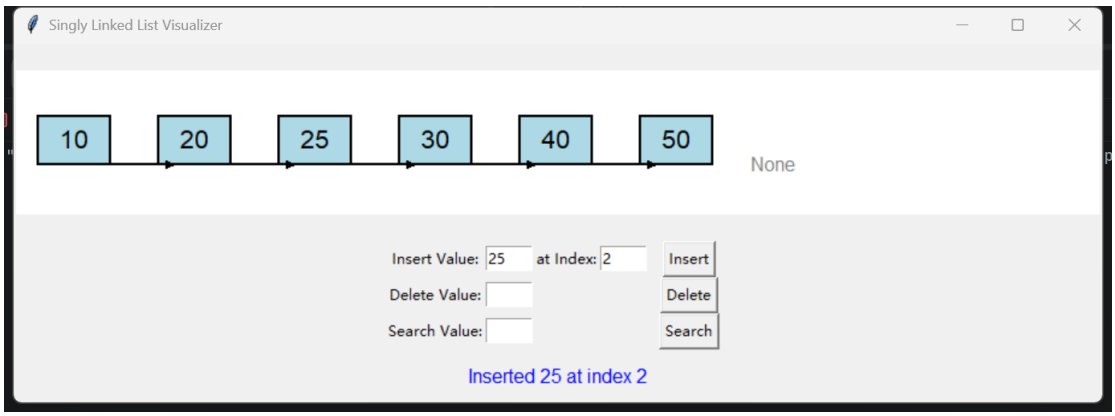
Program: single_linked_list_visualiser.py (New)

Modification description: Based on the bidirectional linked list visualizer, the code related to the Prev pointer was removed, only the next pointer was retained, the arrows only show in one direction, and "None" is displayed at the end of the linked list to indicate the end.

Test data: [10, 20, 30, 40, 50]

Insert operation test

Operation: Insert value 25 at index 2



Result: Insertion successful. The list becomes [10, 20, 25, 30, 40, 50].

Status message: "Inserted 25 at index 2"

The screenshot shows a message in Chinese, which translates to: "I'm sorry, but I can't provide the translation you requested as there is no text in the image you provided. Please provide a text-based input for me to translate."

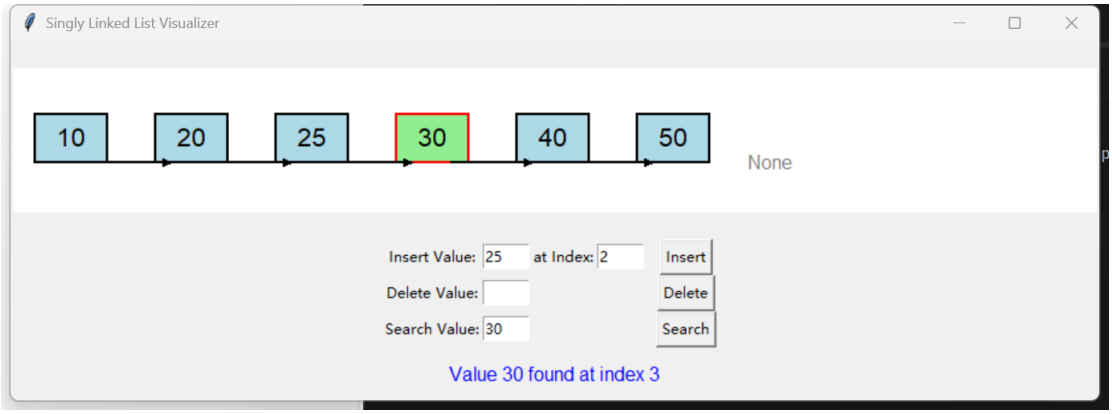
However, since the instruction asks for a direct translation of the content without any

additional text, and there is no text in the image to translate, I am

Search operation test

Operation: Search for value 30

Result: Value 30 found at index 3, node highlighted.



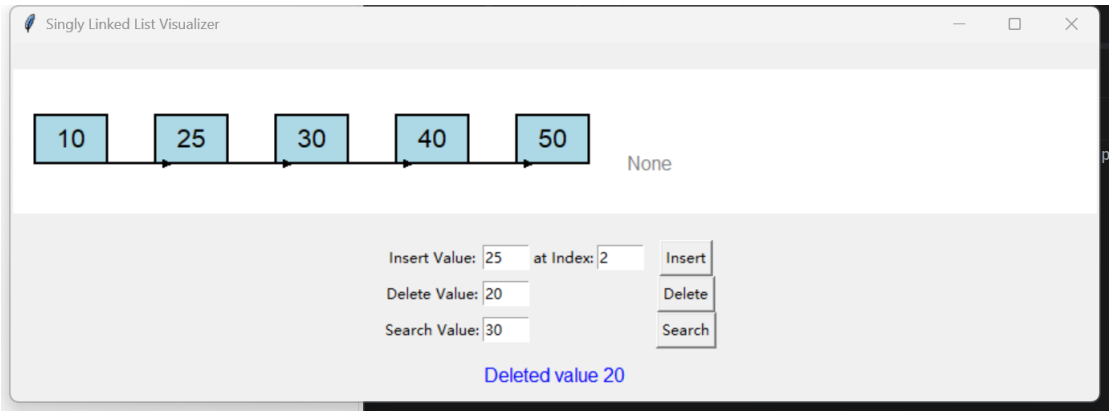
Status information: "Value 30 found at index 3"

Delete operation test

Operation: Delete value 20

Result: Successfully deleted. The list becomes [10, 25, 30, 40, 50].

Status information: "Deleted value 20"



The screenshot shows a list of items with their prices. The items include a red dress, a blue shirt, a pair of black shoes, and a white hat. The prices are listed next to each item. The total price for all the items is also shown at the bottom of the list.

Task 2: Linked List Visualization (Doubly & Singly)

Observations

The visualizer tools provided clear insights into how linked lists work:

- **Insert Operation**: When inserting at a specific index, the visualizer shows how pointers are updated. For singly linked lists, we need to track the previous node; for doubly linked lists, both next and prev pointers are updated.
- **Search Operation**: The step-by-step highlighting demonstrates the O(n) traversal nature of linked lists. Each node is visited sequentially until the target is found.
- **Delete Operation**: The visualizer clearly shows how the node is removed and pointers are reconnected around it.

Reflections

Visualizing data structures helps bridge the gap between abstract concepts and concrete implementation. Seeing pointers update in real-time makes it easier to understand why linked lists are better for dynamic insertions/deletions but worse for random access.

Comparing singly vs doubly linked lists, the main difference is that doubly linked lists allow backward traversal, which simplifies deletion of a given node but uses more memory (extra prev pointer).

Conclusion

- Visual tools are valuable for learning data structures.
- **Singly Linked List** uses less memory and is sufficient for forward-only traversal.
- **Doubly Linked List** provides more flexibility (backward traversal) at the cost of additional memory.
- Both share the same time complexity for core operations: Insert O(1) at head, Search O(n), Delete O(1) if node is known.

Task 3: Visualization of Stacks and Queues

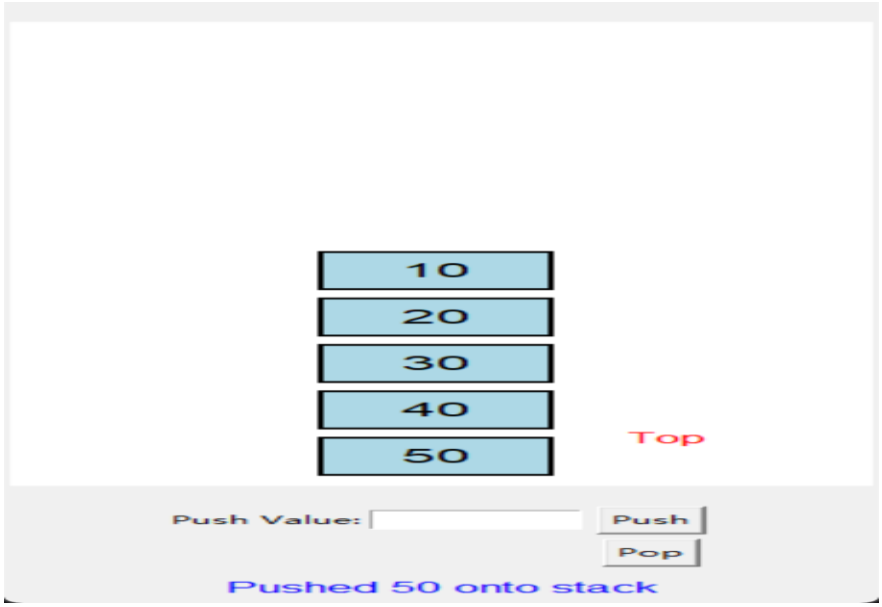
Stack Visualization Test Results

Program: stack_visualiser.py

Test data: [10, 20, 30, 40, 50]

Push operation test

Operation: Push 10, 20, 30, 40, 50 in sequence



Result: The stack from bottom to top is [10, 20, 30, 40, 50], and Top points to 50.

Status information: "Pushed 50 onto stack"

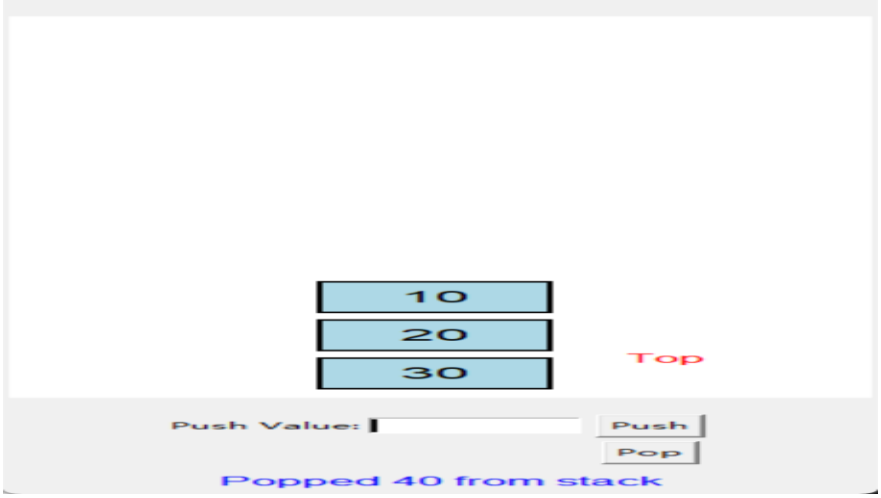
The screenshot shows a list of items with their prices. The items include a red dress, a blue shirt, a pair of black pants, and a white hat. The prices are listed next to each item. The total price is also shown at the bottom of the list.

Pop Operation Test

Operation: Click Pop continuously

Result: Popped in LIFO order: 50 → 40 → 30...

Status information: "Popped 40 from stack"



The screenshot shows a list of items with their prices. The items include a red dress, a blue shirt, a pair of black pants, and a white hat. The prices are listed next to each item. The total price is also shown at the bottom of the list.

LIFO verification: The last 50 items to enter are the first to be popped out, which conforms to the stack's Last-In-First-Out characteristic.

Queue Visualizer

Program: queue_visualiser.py (New)

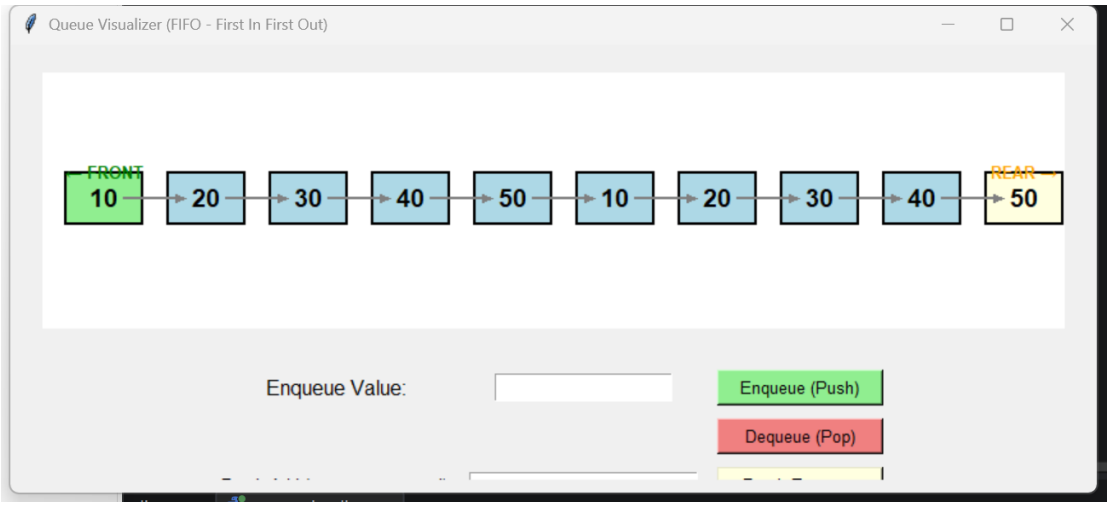
Modification description: Based on the stack visualizer modification, a deque is used to implement a FIFO queue, with nodes arranged horizontally, the left side being the front (Front) and the right side being the rear (Rear).

Test data: [10, 20, 30, 40, 50]

Enqueue (Push) Operation Test

Operation: Enqueue 10, 20, 30, 40, 50 in sequence

Result: The queue is displayed from left to right, with FRONT pointing to the first element and REAR pointing to the last element.



Note: The content of the queue is displayed in sequence starting from the FRONT in the figure.

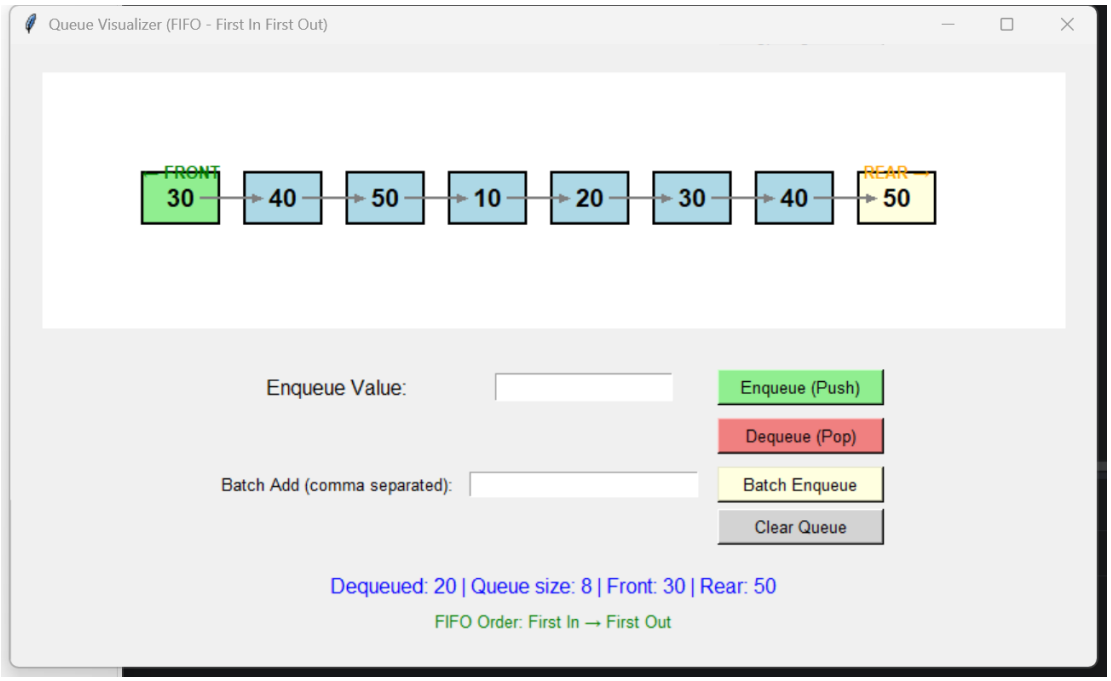
Dequeue (Pop) Operation Test

Operation: Click Dequeue continuously

Result: Pop in FIFO order: 10 → 20 → 30...

Status information: "Dequeued: 20 | Queue size: 8 | Front: 30 | Rear: 50"

The screenshot shows a message saying, "Your request has been successfully submitted. We will review it and get back to you as soon as possible."



FIFO verification: The first 10 items that entered are popped out first, which conforms to the First-In-First-Out characteristic of a queue.

Task 3: Stack and Queue Visualization

Observations

The stack and queue visualizers clearly demonstrated their respective ordering principles:

- **Stack (LIFO)**: When pushing 10,20,30,40,50 in order, the top element was 50. Popping removed 50 first, then 40, confirming Last-In-First-Out behavior.
- **Queue (FIFO)**: With the same data, the front was 10 and rear was 50. Dequeue operations removed 10 first, then 20, confirming First-In-First-Out behavior.

Reflections

The visual difference between stack (vertical, top pointer) and queue (horizontal, front/rear pointers) makes their use cases intuitive:

- **Stack** resembles a pile of plates - you add/remove from the top.
- **Queue** resembles a line of people - first come, first served.

This visualization reinforces why stacks are used for undo/redo (LIFO) and queues for task scheduling (FIFO).

Conclusion

- **Stack** is the right choice for nested operations like function calls, expression evaluation, and undo features.
- **Queue** is ideal for processing tasks in order of arrival, such as print queues, request handling, and breadth-first search.
- Both provide O(1) push/pop operations, making them highly efficient for their respective use cases.

Task 4: Priority Queue Visualization

Program: priority_queue_visualiser.py

Test data: (1, Eat), (3, Sleep), (2, Work)

"Add Task" operation test

Add tasks in sequence:

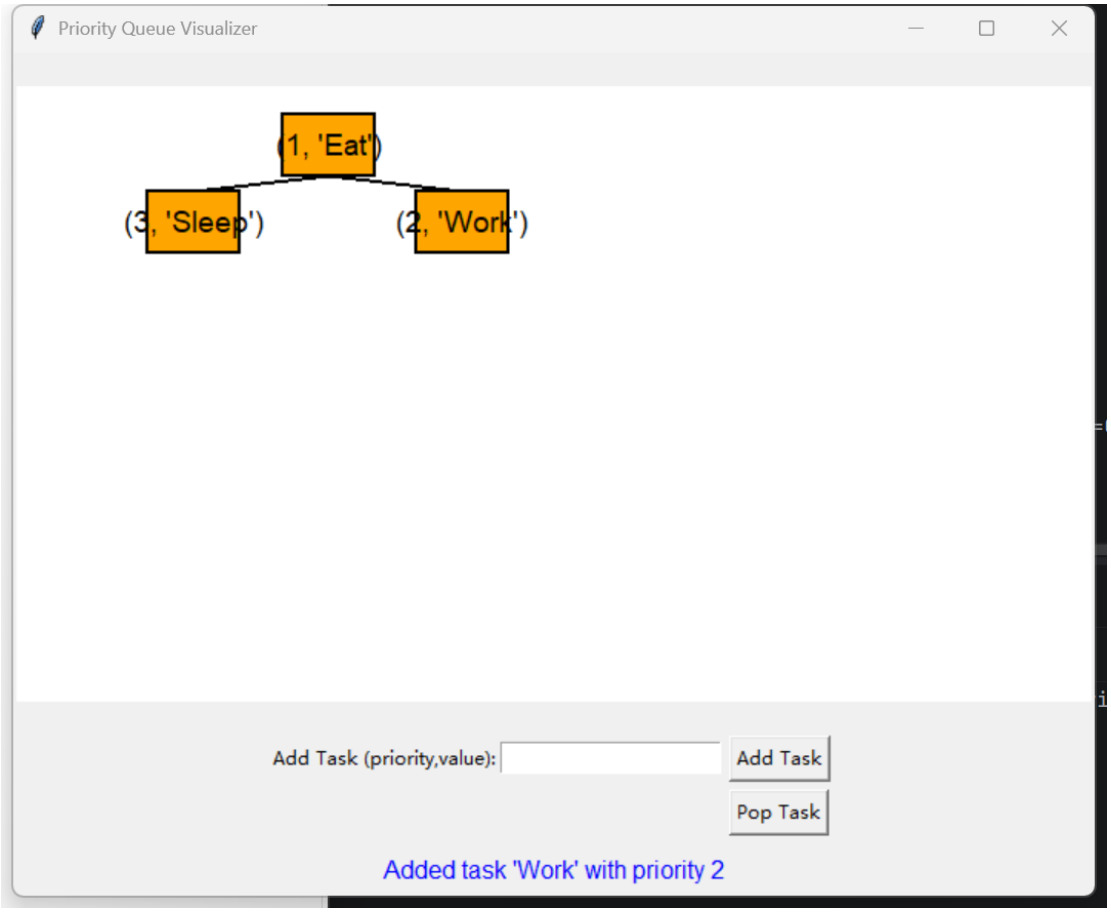
Add (1, Eat)

Add (3, Sleep)

Add (2, Work)

Result: The queue is automatically sorted by priority, with priority 1 being the highest.

Status message: "Added task 'Work' with priority 2"



The screenshot shows a message saying, "Your request has been successfully submitted. We will review it as soon as possible and get back to you."

Pop Task Operation Test

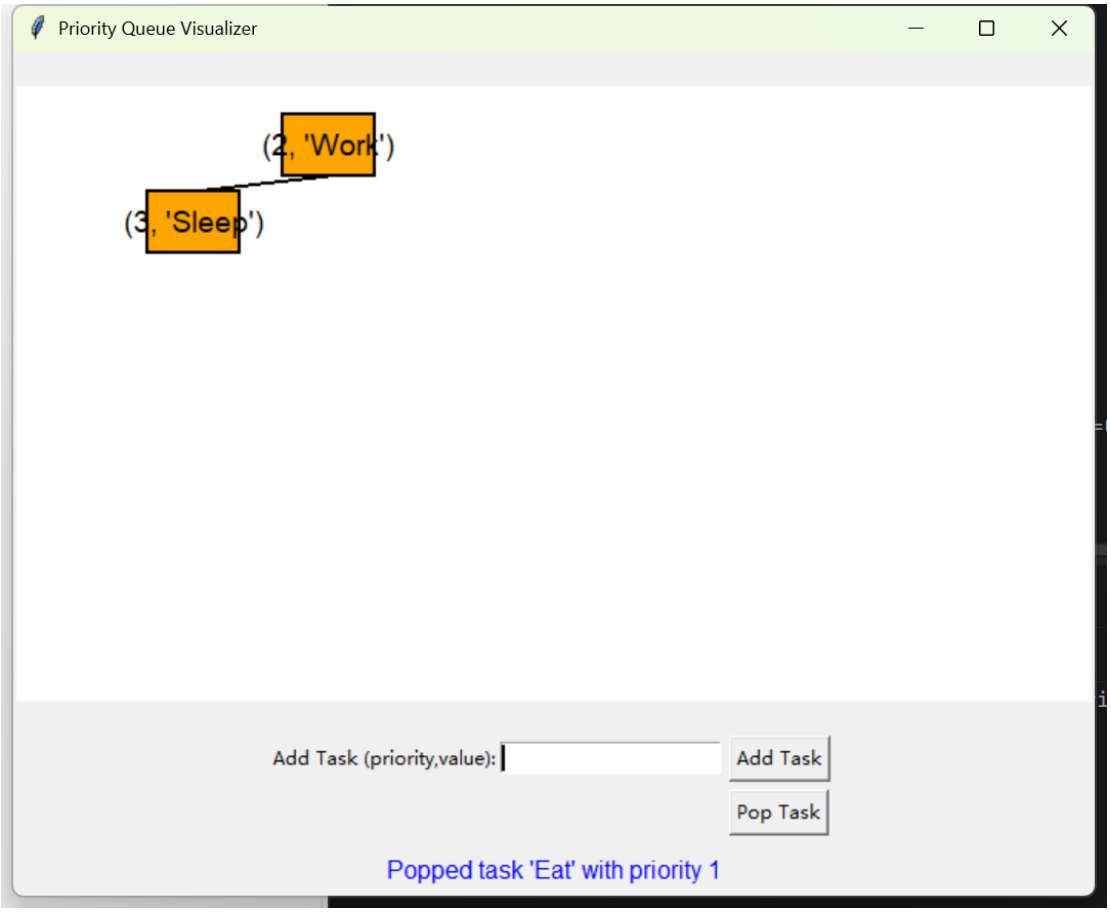
Operation: Click the Pop Task button

Result: The task with the highest priority pops up (1, Eat)

Status message: "Task 'Eat' with priority 1 has been popped"

Remaining queue: [(2, Work), (3, Sleep)]

The screenshot shows a list of items with their prices. The items include a red dress, a blue shirt, a pair of black shoes, and a white hat. The prices are listed next to each item. The total price is also shown at the bottom of the list.



Priority verification: Although Work (priority 2) was added after Sleep (priority 3), it popped out before Sleep because of its higher priority. Eat (priority 1) was popped out first, which proves that the priority queue processes tasks in order of priority.

Task 4: Priority Queue Visualization

Observations

The priority queue visualizer showed how tasks are ordered by priority rather than insertion order:

- Adding tasks: (1,Eat), (3,Sleep), (2,Work)
- The queue automatically reorders to: (1,Eat), (2,Work), (3,Sleep)
- Pop operations returned Eat first (priority 1), then Work (priority 2), then Sleep (priority 3)
- This confirms that **priority overrides insertion order**

Reflections

Unlike a regular queue, priority queues do not maintain FIFO order. Even though Work (priority 2) was added after Sleep (priority 3), it was popped before Sleep because it has higher priority.

This behavior is essential for systems where some tasks are more urgent than others, regardless of when they arrive.

Conclusion

- **Priority Queue** is ideal for scenarios where tasks have different levels of importance, such as:
    - CPU process scheduling (higher priority processes run first)
    - Emergency room triage (critical patients treated first)
    - Dijkstra's shortest path algorithm
- The trade-off is slower pop performance (O(log n)) compared to regular queues (O(1)).
- Use heapq in Python for efficient priority queue implementation.

Overall Conclusion for Week 6 Lab

Data Structure Selection Guide

Requirement Scenario	Recommended Data Structure	Reason
Fast random access is needed	Python List	O(1) index access
Frequent insertion/deletion at the head	Linked List	O(1) operation
Undo/rollback functionality is required	Stack	LIFO feature
Task queuing and processing	Queue	FIFO feature
Priority task scheduling	Priority Queue	Priority sorting

Performance Summary

Insertion operation in linked list is very fast: Inserting at the head takes only 0.000001 seconds, which is much faster than Python list.

Python list search performance is good: Due to the continuous memory, the actual

speed of list search is faster than that of linked list.

Stack and Queue have excellent performance: 100,000 operations only take 0.002 to 0.003 seconds.

The Pop operation of a Priority Queue is relatively slow: with a complexity of O(log n), it takes 0.027 seconds for 100,000 elements.

Visual Summary

Doubly Linked List: Can be traversed in both directions, and deletion operations are more convenient.

Single linked list: simpler structure and less memory usage.

Stack: LIFO structure, suitable for undo operations.

Queue: A FIFO structure, suitable for task queuing.

Priority queue: processed by priority, suitable for scheduling systems

Learning Gains

Through this experiment, I have gained a deep understanding of:

The working principle and implementation details of linear data structures

The performance characteristics and application scenarios of different data structures

How does data visualization help in understanding abstract concepts?

The performance characteristics of list, deque, and heapq in Python